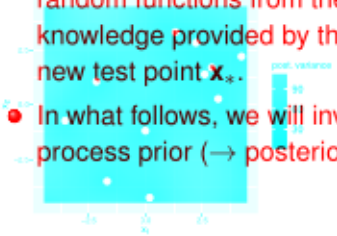
GAUSSIAN POSTERIOR PROCESS AND PREDICTION

Introduction to Machine Learning

Gaussian Process Prediction

- So far, we have learned how to **sample** from a GP prior.
- However, most of the time, we are not interested in drawing random functions from the prior. Instead, we usually like to use the knowledge provided by the training data to predict values of f at a new test point $\mathbf{x}_*$.
- In what follows, we will investigate how to update the Gaussian process prior ($\rightarrow$ posterior process) and how to make predictions.



Learning goals

- Know how to derive the posterior process
- GPs are interpolating and spatial models
- Model noise via a nugget term

Gaussian Posterior Process and Prediction

- However, most of the time, we are not interested in drawing random functions from the prior. Instead, we usually like to use the knowledge provided by the training data to predict values of f at a new test point $\mathbf{x}_*$.
- In what follows, we will investigate how to update the Gaussian process prior ($\rightarrow$ posterior process) and how to make predictions.

POSTERIOR PROCESS

- Let us now distinguish between observed training inputs, also denote by a design matrix $\mathbf{X}$, and the corresponding observed values

$$\boldsymbol{f} = \left[f\left(\mathbf{x}^{(1)}\right), \ldots, f\left(\mathbf{x}^{(n)}\right)\right]$$

Gaussian Posterior Process and Prediction

and one single **unobserved test point** $\mathbf{x}_*$ with $f_* = f(\mathbf{x}_*)$.

- We now want to infer the distribution of $f_* | \mathbf{x}_*, \boldsymbol{X}, \boldsymbol{f}$.

$$f_* = f(\mathbf{x}_*)$$

- Assuming a zero-mean GP prior $\mathcal{GP}(\mathbf{0}, k(\mathbf{x}, \mathbf{x}'))$ we know

$$\begin{bmatrix} \boldsymbol{f} \\ f_* \end{bmatrix} \sim \mathcal{N}\left(\mathbf{0}, \begin{bmatrix} \mathbf{K} & \boldsymbol{k}_* \\ \boldsymbol{k}_*^T & \boldsymbol{k}_{**} \end{bmatrix}\right).$$

Here, $\mathbf{K} = \left(k\left(\mathbf{x}^{(i)}, \mathbf{x}^{(j)}\right)\right)_{i,j}$, $\boldsymbol{k}_* = \left[k\left(\mathbf{x}_*, \mathbf{x}^{(1)}\right), \ldots, k\left(\mathbf{x}_*, \mathbf{x}^{(n)}\right)\right]$ and $\boldsymbol{k}_{**} = k(\mathbf{x}_*, \mathbf{x}_*)$.

GP PREDICTION: TWO POINTS

Let us visualize this by a simple example:

- Assume we observed a single training point $\mathbf{x} = -0.5$, and want to make a prediction at a test point $\mathbf{x}_* = 0.5$.
- Under a zero-mean GP with $k(\mathbf{x}, \mathbf{x}') = \exp\left(-\frac{1}{2}\|\mathbf{x} - \mathbf{x}'\|^2\right)$, we compute the cov-matrix:

$$\begin{bmatrix} f \\ f_* \end{bmatrix} \sim \mathcal{N}\left(\mathbf{0}, \begin{bmatrix} 1 & 0.61 \\ 0.61 & 1 \end{bmatrix}\right).$$

- Assume that we observe the point $f(\mathbf{x}) = 1$.
- We compute the posterior distribution:

$$f_* \mid \mathbf{x}_*, \mathbf{x}, f \sim \mathcal{N}(\boldsymbol{k}_*^T \mathbf{K}^{-1} f, k_{**} - \boldsymbol{k}_*^T \mathbf{K}^{-1} \boldsymbol{k}_*)$$

$$\begin{bmatrix} f \\ f_* \end{bmatrix} \sim \mathcal{N}\left(\mathbf{0}, \begin{bmatrix} \mathbf{K} & \boldsymbol{k}_* \\ \boldsymbol{k}_*^T & k_{**} \end{bmatrix}\right)$$

$$\sim \mathcal{N}(0.61 \cdot 1 \cdot 1, 1 - 0.61 \cdot 1 \cdot 0.61)$$

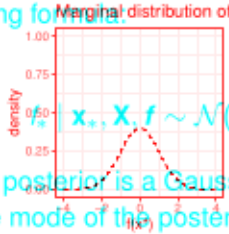
$$\sim \mathcal{N}(0.61, 0.6279)$$

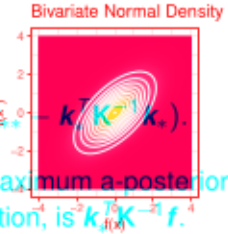
- The MAP-estimate for $\mathbf{x}_*$ is $f(\mathbf{x}_*) = 0.61$, and the uncertainty estimate is 0.6279.

GP PREDICTION: TWO POINTS

Shown is the bivariate normal density, and the respective marginals.

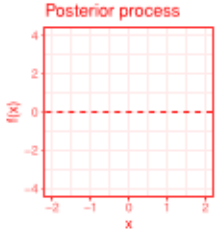
- Given that f is a GP and we apply the general rule for condition $^{(*)}$ of Gaussian random variables and obtain the following formula:

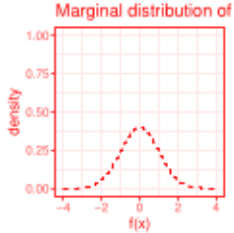




$$f_* \mid \mathbf{x}_*, \mathbf{X}, \boldsymbol{f} \sim \mathcal{N}(\boldsymbol{k}_*^T \mathbf{K}^{-1} \boldsymbol{f}, k_{**} - \boldsymbol{k}_*^T \mathbf{K}^{-1} \boldsymbol{k}_*).$$

- As the posterior is a Gaussian, the maximum a-posteriori estimate, i.e. the mode of the posterior distribution, is $\boldsymbol{k}_*^T \mathbf{K}^{-1} \boldsymbol{f}$.





GP PREDICTION: TWO POINTS

Assume we observed $f(\mathbf{x}) = 1$ for the training point $\mathbf{x} = -0.5$.

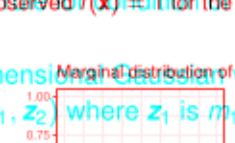
(a) Generalize formula for conditional of Gaussian random variables:

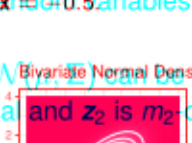
If the m-dimensional Gaussian vector $\boldsymbol{z} \sim \mathcal{N}(\mu, \Sigma)$ can be partitioned with $\boldsymbol{z} = (\boldsymbol{z}_1, \boldsymbol{z}_2)$ where $\boldsymbol{z}_1$ is m_1-dimensional and $\boldsymbol{z}_2$ is m_2-dimensional, and:

$$(\mu_1, \mu_2), \quad \Sigma = \begin{pmatrix} \Sigma_{11} & \Sigma_{12} \\ \Sigma_{21} & \Sigma_{22} \end{pmatrix}$$

then the conditional distribution of $\boldsymbol{z}_2 \mid \boldsymbol{z}_1 = \boldsymbol{a}$ is a multivariate normal

$$\mathcal{N}\left(\mu_2 + \Sigma_{21}\Sigma_{11}^{-1}(\boldsymbol{a} - \mu_1), \Sigma_{22} - \Sigma_{21}\Sigma_{11}^{-1}\Sigma_{12}\right)$$



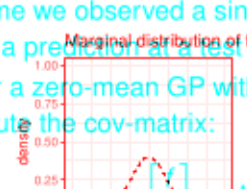


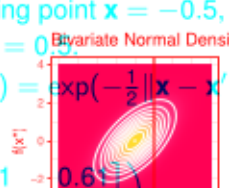
GP PREDICTION: TWO POINTS

We condition the Gaussian on $f(\mathbf{x}) = 1$.

Let's illustrate this with a simple example:

- Assume we observed a single training point $\mathbf{x} = -0.5$, and want to make a prediction at a test point $\mathbf{x}_* = 0.5$.
- Under a zero-mean GP with $k(\mathbf{x}, \mathbf{x}') = \exp(-\frac{1}{2}\|\mathbf{x} - \mathbf{x}'\|^2)$, we compute the cov-matrix:





$$\begin{bmatrix} f \\ f_* \end{bmatrix} \sim \mathcal{N}\left(\mathbf{0}, \begin{bmatrix} 1 & 0.61 \\ 0.61 & 1 \end{bmatrix}\right).$$

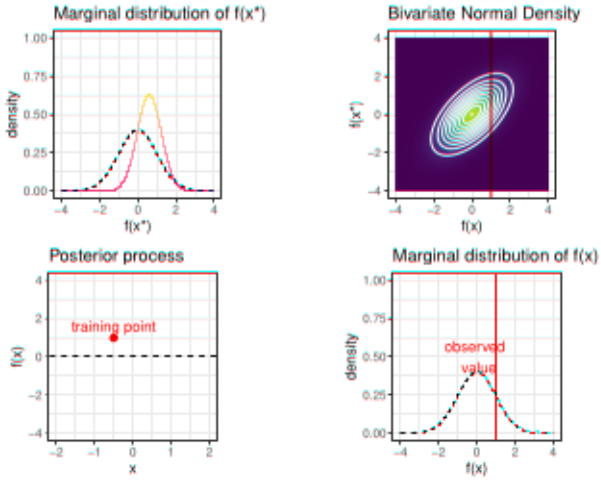
- Assume that we observe the point $f(\mathbf{x}) = 1$.
- We compute the posterior distribution:

$$\begin{aligned} f_* \mid \mathbf{x}_*, \mathbf{x}, f &\sim \mathcal{N}(\boldsymbol{k}_*^T \mathbf{K}^{-1} f, k_{**} - \boldsymbol{k}_*^T \mathbf{K}^{-1} \boldsymbol{k}_*) \\ &\sim \mathcal{N}(0.61 \cdot 1 \cdot 1, 1 - 0.61 \cdot 1 \cdot 0.61) \\ &\sim \mathcal{N}(0.61, 0.6279). \end{aligned}$$

- The MAP-estimate for $\mathbf{x}_*$ is $f(\mathbf{x}_*) = 0.61$, and the uncertainty estimate is 0.6279.

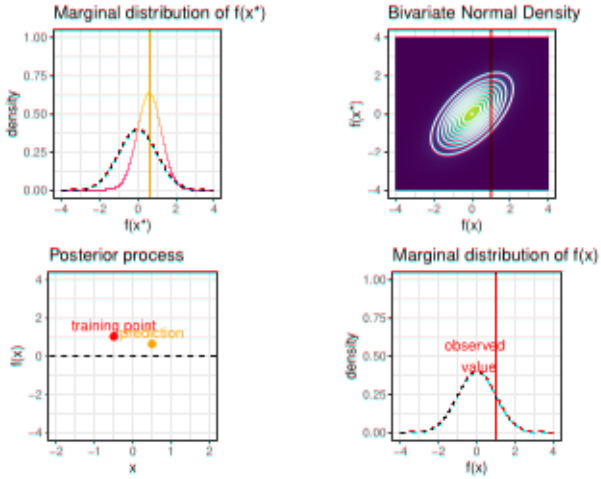
GP PREDICTION: TWO POINTS

We compute the posterior distribution of $f(\mathbf{x}^*)$ given that $f(\mathbf{x}) = 1$.



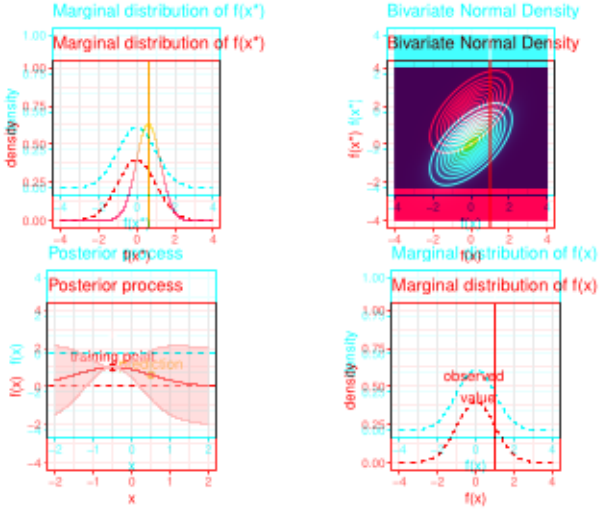
GP PREDICTION: TWO POINTS

A possible predictor for $f(x)$ at $\mathbf{x}_*$ is the MAP of the posterior distribution.



GP PREDICTION: TWO POINTS

We can do this for different values x_*, and show the respective mean (grey line) and standard deviations (grey area is mean $\pm 2 \cdot$ posterior standard deviation).



POSTERIOR PROCESS POINTS

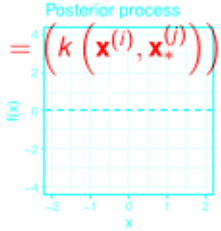
We compute the posterior distribution of $f(\mathbf{x}_*)$ given that $f(\mathbf{x}) = \mathbf{y}$

- We can generalize the formula for the posterior process for multiple unobserved test points:

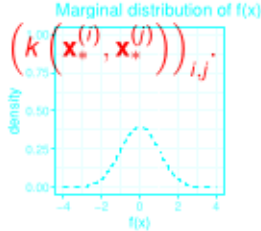
$$\boldsymbol{f}_* = \left[f\left(\mathbf{x}_*^{(1)}\right), \ldots, f\left(\mathbf{x}_*^{(m)}\right)\right].$$

- Under a zero-mean Gaussian process, we have

$$\begin{bmatrix} \boldsymbol{f} \\ \boldsymbol{f}_* \end{bmatrix} \sim \mathcal{N}\left(\mathbf{0}, \begin{bmatrix} \mathbf{K} & \mathbf{K}_* \\ \mathbf{K}_*^T & \mathbf{K}_{**} \end{bmatrix}\right),$$

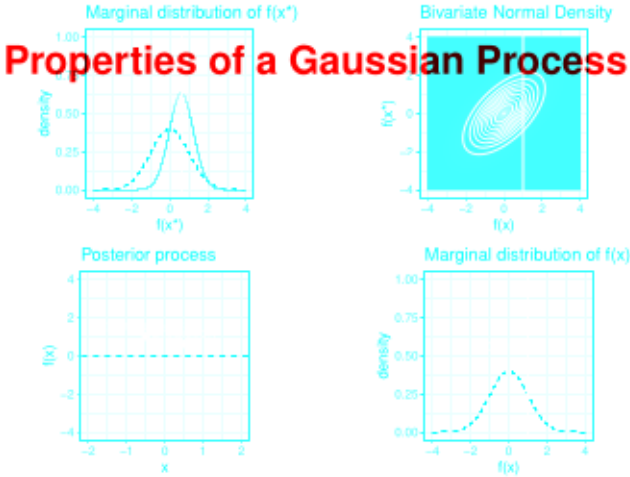


with $\mathbf{K}_* = \left(k\left(\mathbf{x}^{(i)}, \mathbf{x}_*^{(j)}\right)\right)_{i,j}$, $\mathbf{K}_{**} = \left(k\left(\mathbf{x}_*^{(i)}, \mathbf{x}_*^{(j)}\right)\right)_{i,j}$.



GP PREDICTION: TWO POINTS

A possible predictor for f at $\mathbf{x}_*$ is the MAP of the posterior distribution.

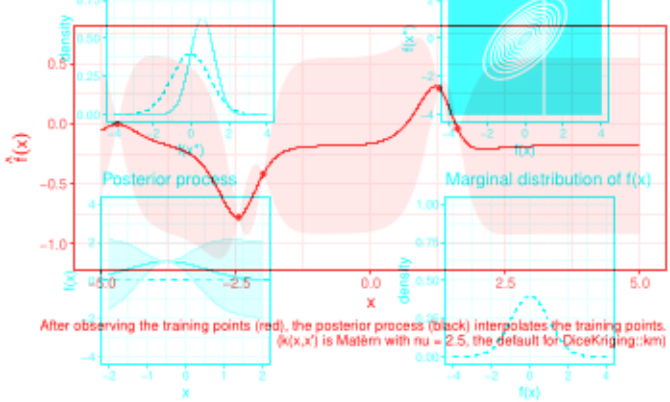


GP AS INTERPOLATOR

The "prediction" for a training point $\mathbf{x}^{(i)}$ is the exact function value $f(\mathbf{x}^{(i)})$.

$$\boldsymbol{f} \mid \mathbf{X}, \boldsymbol{f} \sim \mathcal{N}(\mathbf{K}\mathbf{K}^{-1}\boldsymbol{f}, \mathbf{K} - \mathbf{K}^T\mathbf{K}^{-1}\mathbf{K}) = \mathcal{N}(\boldsymbol{f}, \mathbf{0}).$$

Thus, a Gaussian process is a function **interpolator**.



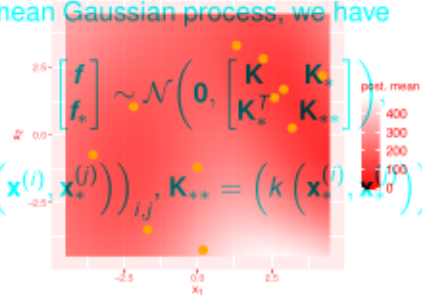
After observing the training points (red), the posterior process (black) interpolates the training points.
(k(x,x') is Matérn with nu = 2.5, the default for DiceKriging::km)

GP AS A SPATIAL MODEL

- The correlation among two outputs depends on distance of the corresponding input points $\mathbf{x}$ and $\mathbf{x}'$ (e.g. Gaussian covariance kernel $k(\mathbf{x}, \mathbf{x}') = \exp\left(\frac{-\|\mathbf{x}-\mathbf{x}'\|^2}{2l^2}\right)$).
- Hence, close data points with high spatial similarity $k(\mathbf{x}, \mathbf{x}')$ enter into more strongly correlated predictions: $f_* = \left[f\left(\mathbf{x}_*^{(1)}\right) \dots f\left(\mathbf{x}_*^{(m)}\right)\right] = \mathbf{k}_*^T \mathbf{K}^{-1} \mathbf{f}$ ($\mathbf{k}_* := \left(k(\mathbf{x}, \mathbf{x}^{(1)}), \dots, k(\mathbf{x}, \mathbf{x}^{(n)})\right)$).
- Under a zero-mean Gaussian process, we have

$$\begin{bmatrix} \mathbf{f} \\ \mathbf{f}_* \end{bmatrix} \sim \mathcal{N}\left(\mathbf{0}, \begin{bmatrix} \mathbf{K} & \mathbf{K}_* \\ \mathbf{K}_*^T & \mathbf{K}_{**} \end{bmatrix}\right)$$

with $\mathbf{K}_* = \left(k\left(\mathbf{x}^{(i)}, \mathbf{x}_*^{(j)}\right)\right)_{i,j}$, $\mathbf{K}_{**} = \left(k\left(\mathbf{x}_*^{(i)}, \mathbf{x}_*^{(j)}\right)\right)_{i,j}$.



Example: Posterior mean of a GP that was fitted with the Gaussian covariance kernel with $l = 1$.

POSTERIOR PROCESS

- Similar to the single test point situation, to get the posterior distribution, we exploit the general rule of conditioning for Gaussians:

Noisy Gaussian Process

$$f_* \mid X_*, X, y \sim \mathcal{N}(K_* K^{-1} y, K_{**} - K_* K^{-1} K_*).$$

- This formula enables us to talk about correlations among different test points and sample functions from the posterior process.

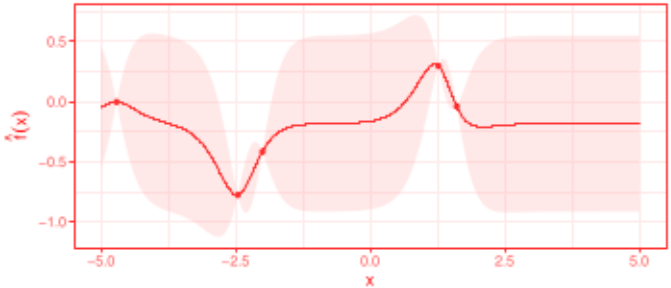
NOISY GAUSSIAN PROCESS

- So far, we implicitly assumed that we had access to the true function value $f(\mathbf{x})$.
- For the squared exponential kernel, for example, we have

Properties of a Gaussian Process

$$\text{Cov}\left(f(\mathbf{x}^{(i)}), f(\mathbf{x}^{(i)})\right) = 1.$$

- As a result, the posterior Gaussian process is an interpolator:



After observing the training points (red), the posterior process (black) interpolates the training points.
(k(x,x') is Matérn with nu = 2.5, the default for DiceKriging::km)

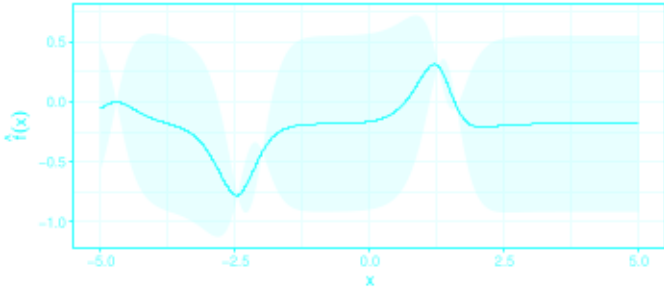
GP AS INTERPOLATOR

The "prediction" for a training point $\mathbf{x}^{(i)}$ is the exact function value $f\left(\mathbf{x}^{(i)}\right)$

$$\boldsymbol{f} \mid \mathbf{X}, \boldsymbol{f} \sim \mathcal{N}(\mathbf{K}\mathbf{K}^{-1}\boldsymbol{f}, \mathbf{K} - \mathbf{K}^T\mathbf{K}^{-1}\mathbf{K}) = \mathcal{N}(\boldsymbol{f}, \mathbf{0}).$$

Decision Theory for Gaussian Processes

Thus, a Gaussian process is a function **interpolator**.



After observing the training points (red), the posterior process (black) interpolates the training points.
(k(x,x') is Matérn with nu = 2.5, the default for DiceKriging::km)

RISK MINIMIZATION FOR GAUSSIAN PROCESSES

- The correlation among two outputs depends on distance of the corresponding input points $\mathbf{x}$ and $\mathbf{x}'$ (e.g. Gaussian covariance kernel $k(\mathbf{x}, \mathbf{x}') = \exp\left(-\frac{\|\mathbf{x}-\mathbf{x}'\|^2}{2l^2}\right)$)
- Hence, close data points with high spatial similarity $k(\mathbf{x}, \mathbf{x}')$ enter into more strongly correlated predictions $f(\mathbf{x}), f(\mathbf{x}')$.

In machine learning, we learned about risk minimization. We usually choose a loss function and minimize the empirical risk

$$\mathcal{R}_{\text{emp}}(f) := \sum_{i=1}^{n} L\left(y^{(i)}, f\left(\mathbf{x}^{(i)}\right)\right)$$

as an approximation to the theoretical risk

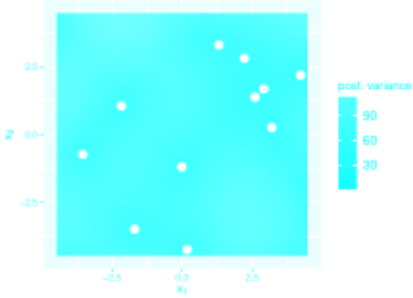
$$\mathcal{R}(f) := \mathbb{E}_{xy}[L(y, f(\mathbf{x}))] = \int L(y, f(\mathbf{x}))\, \mathrm{d}\mathbb{P}_{xy}.$$

- How does the theory of Gaussian processes fit into this theory?
- What if we want to make a prediction which is optimal w.r.t. a certain loss function?

Example: Posterior mean of a GP that was fitted with the Gaussian covariance kernel with $l = 1$.

GP AS A SPATIAL MODEL

- Posterior uncertainty increases if the new data points are far from the design points.
- The uncertainty is minimal at the design points, since the posterior variance is zero at these points.



Example (continued): Posterior variance.

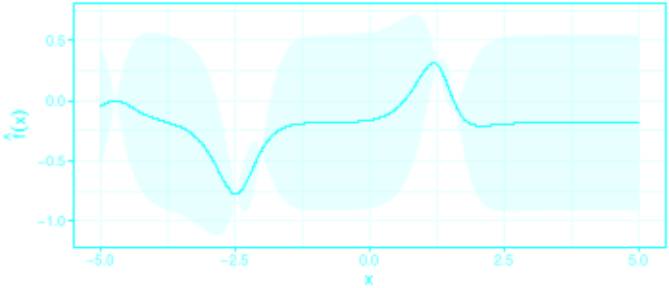
Noisy Gaussian Process

NOISY GAUSSIAN PROCESS

- So far, we implicitly assumed that we had access to the true function value $f(\mathbf{x})$.
- For the squared exponential kernel, for example, we have

$$\text{Cov}\left(f(\mathbf{x}^{(i)}), f(\mathbf{x}^{(i)})\right) = 1.$$

- As a result, the posterior Gaussian process is an interpolator:



After observing the training points (red), the posterior process (black) interpolates the training points. (k(x,x') is Matérn with nu = 2.5, the default for DiceKriging::km)

NOISY GAUSSIAN PROCESS

- In reality, however, this is often not the case.
- We often only have access to a noisy version of the true function value

$$y = f(\mathbf{x}) + \epsilon, \epsilon \sim \mathcal{N}\left(0, \sigma^2\right).$$

- Let us still assume that $f(\mathbf{x})$ is a Gaussian process.
- Then,

$$\begin{aligned} & \mathrm{Cov}(y^{(i)}, y^{(j)}) = \mathrm{Cov}\left(f\left(\mathbf{x}^{(i)}\right) + \epsilon^{(i)}, f\left(\mathbf{x}^{(j)}\right) + \epsilon^{(j)}\right) \\ = \; & \mathrm{Cov}\left(f\left(\mathbf{x}^{(i)}\right), f\left(\mathbf{x}^{(j)}\right)\right) + 2 \cdot \mathrm{Cov}\left(f\left(\mathbf{x}^{(i)}\right), \epsilon^{(j)}\right) + \mathrm{Cov}\left(\epsilon^{(i)}, \epsilon^{(j)}\right) \\ = \; & k\left(\mathbf{x}^{(i)}, \mathbf{x}^{(j)}\right) + \sigma^2 \delta_{ij}. \end{aligned}$$

- σ^2 is called **nugget**.

NOISY GAUSSIAN PROCESS

- Let us now derive the predictive distribution for the case of noisy observations.
- The prior distribution of y, assuming that f is modeled by a Gaussian process is then

$$\boldsymbol{y} = \begin{pmatrix} y^{(1)} \\ y^{(2)} \\ \vdots \\ y^{(n)} \end{pmatrix} \sim \mathcal{N}\left(\boldsymbol{m}, \boldsymbol{K} + \sigma^2 \boldsymbol{I}_n\right),$$

with

$$\mathbf{m} := \left(m\left(\mathbf{x}^{(i)}\right)\right)_i, \quad \mathbf{K} := \left(k\left(\mathbf{x}^{(i)}, \mathbf{x}^{(j)}\right)\right)_{i,j}.$$

NOISY GAUSSIAN PROCESS

- We distinguish again between
  - observed training points $\mathbf{X}, \mathbf{y}$, and
  - unobserved test inputs $\mathbf{X}_*$ with unobserved values $\boldsymbol{f}_*$

  and get

$$\begin{bmatrix} \boldsymbol{y} \\ \boldsymbol{f}_* \end{bmatrix} \sim \mathcal{N}\left(\mathbf{0}, \begin{bmatrix} \mathbf{K} + \sigma^2 \boldsymbol{I}_n & \mathbf{K}_* \\ \mathbf{K}_*^T & \mathbf{K}_{**} \end{bmatrix}\right).$$

NOISY GAUSSIAN PROCESS

- Similarly to the noise-free case, we condition according to the rule of conditioning for Gaussians to get the posterior distribution for the test outputs $\boldsymbol{f}_*$ at $\mathbf{X}_*$:

$$\boldsymbol{f}_* \mid \mathbf{X}_*, \mathbf{X}, \boldsymbol{y} \sim \mathcal{N}(\boldsymbol{m}_{\text{post}}, \boldsymbol{K}_{\text{post}}).$$

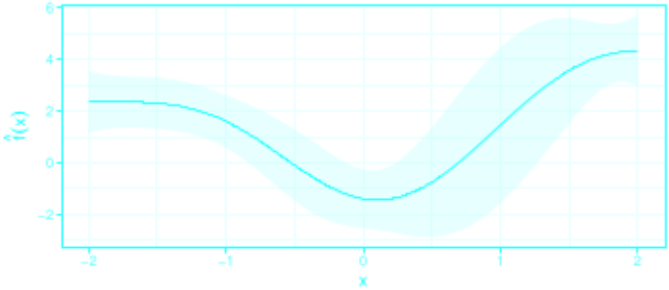
with

$$\begin{aligned} \boldsymbol{m}_{\text{post}} &= \mathbf{K}_*^T \left(\mathbf{K} + \sigma^2 \cdot \boldsymbol{I}\right)^{-1} \boldsymbol{y} \\ \boldsymbol{K}_{\text{post}} &= \mathbf{K}_{**} - \mathbf{K}_*^T \left(\mathbf{K}^{-1} + \sigma^2 \cdot \boldsymbol{I}\right) \mathbf{K}_*, \end{aligned}$$

- This converts back to the noise-free formula if $\sigma^2 = 0$.

NOISY GAUSSIAN PROCESS

- The noisy Gaussian process is not an interpolator any more.
- A larger nugget term leads to a wider "band" around the observed training points.
- The nugget term is estimated during training.



After observing the training points (red), we have a nugget–band around the oberved points.
($k(x,x')$ is the squared exponential)

Decision Theory for Gaussian Processes

RISK MINIMIZATION FOR GAUSSIAN PROCESSES

In machine learning, we learned about risk minimization. We usually choose a loss function and minimize the empirical risk

$$\mathcal{R}_{\text{emp}}(f) := \sum_{i=1}^{n} L\left(y^{(i)}, f\left(\mathbf{x}^{(i)}\right)\right)$$

as an approximation to the theoretical risk

$$\mathcal{R}(f) := \mathbb{E}_{xy}[L(y, f(\mathbf{x}))] = \int L(y, f(\mathbf{x}))\, \mathrm{d}\mathbb{P}_{xy}.$$

- How does the theory of Gaussian processes fit into this theory?
- What if we want to make a prediction which is optimal w.r.t. a certain loss function?

RISK MINIMIZATION FOR GAUSSIAN PROCESSES

- The theory of Gaussian process gives us a posterior distribution

$$p(y \mid \mathcal{D})$$

- If we now want to make a prediction at a test point $\boldsymbol{x}_*$, we approximate the theoretical risk in a different way, by using the posterior distribution:

$$\mathcal{R}(y_* \mid \boldsymbol{x}_*) \approx \int L(\tilde{y}_*, y_*) p(\tilde{y}_* \mid \boldsymbol{x}_*, \mathcal{D}) d\tilde{y}_*.$$

- The optimal prediciton w.r.t the loss function is then:

$$\hat{y}_* | \boldsymbol{x}_* = \arg\min_{y_*} \mathcal{R}(y_* \mid \boldsymbol{x}_*).$$